

This DSL is designed as a minimal, readable language for **mathematical computations**, particularly focused on number theory operations such as greatest common divisor (GCD / `cmmdc`), least common multiple (LCM / `cmmmc`), and power sums of integers. Its primary **use case** is to allow learners, teachers, and script writers to quickly express and compute math problems interactively or in batch scripts. By combining simple input/output commands, domain-specific operators (**`^^`**, **`%%`**, **`@@`**), and control structures, it enables writing clear and maintainable programs without requiring the full complexity of a general-purpose language.

The choice of statements in the DSL reflects its focus on usability and clarity. **INPUT** and **OUTPUT** provide basic interactivity, making it easy to read values from the user and display results. The **DEF ... BEGIN ... END** structure lets users define reusable, named functions, with **RETURN** allowing those functions to produce computed results. Conditionals with **IF ... THEN ... ELSE ... END** enable decisions and comparisons, essential for more dynamic scripts. At the heart of the DSL are the expressions that use standard math operators as well as custom domain-specific operators — **`^^`** for GCD, **`%%`** for LCM, and **`@@`** for power sum — making these calculations both intuitive and succinct.

This DSL is well-suited to a variety of settings: classrooms (to teach math concepts through programming), quick scripts for number theory tasks (such as calculating properties of integers), or as a lightweight scripting tool embedded in larger systems. The minimal syntax lowers the barrier to entry for non-programmers while still offering enough expressiveness for meaningful computation. Its statements were chosen to strike a balance between simplicity, expressiveness, and focus on the domain of integer computations and comparisons.